



Scrye Companion Design Notes

Why the mobile companion is built the way it is:
protocol, threading, security.

Internal — design rationale, not a user manual · last updated 2026-08-02

Status: Design / proposal · **Last updated:** 2026-08-02

This document describes how Scrye becomes a "full-circle" MUD platform: the desktop client keeps doing the hard work (Telnet, ANSI, triggers, scripts, maps, state) and a mobile — or browser — companion acts as a thin, touch-friendly frontend over a secure connection to the PC.

It is a decision record, not a task list. It captures *why* the architecture is shaped the way it is, what already exists in the codebase to build on, and the order in which the pieces should be built.

1. The core idea

```
MUD server
  ↳ Telnet / MCCP / GMCP / MIP
Desktop client (Scrye.App on the PC) ← single source of truth
  ↳ secure WebSocket (LAN / VPN / relay)
Companion frontend (browser first, native later)
```

The desktop remains responsible for everything stateful and long-lived: the Telnet connection, ANSI/MXP parsing, triggers and aliases, scripts and plugins, timers, logging, variables, maps, character state, and reconnection. The companion receives *already-processed* information (output lines, character state, room, map data, chat, available commands, connection state) and sends back *actions* (send command, run alias, press a shortcut, switch session, start a route, pause a script).

Why not remote desktop?

Google Remote Desktop / VNC streams the desktop's pixels to the phone: tiny text, desktop-sized buttons, awkward zoom, a virtual mouse. A dedicated companion sends *structured data* instead of a video stream, so the phone renders a proper mobile UI with its own fonts, layout, and touch controls — while the PC still does all the work.

Why not a full mobile MUD client?

Because a phone **cannot reliably hold a Telnet socket alive in the background** — iOS and Android suspend or kill backgrounded network apps. The PC can. Making the desktop the always-on host and the phone a resumable frontend works *with* the mobile platforms instead of against them. This single fact drives most of the rest of the design (sequence-numbered resume, snapshot-on-reconnect, "PC stays connected while the phone sleeps").

2. What already exists (build on this, don't rebuild it)

Scrye is further along toward this than a greenfield plan assumes. Three existing subsystems do most of the work the companion needs:

An internal event stream is effectively already there. `MudSession` emits events, `WorldViewModel` already fans output and state into the HUD, and there is a shared state tree — `character.*`, `enemy.*`, `vik.*`, `plugin.<id>.*` — that plugins push into with `scrye.setState` and observe with `scrye.watch`. A `state.update` companion message is close to a direct serialization of one node in that tree. The companion server is mostly a *tap* on flows that already exist, not new plumbing.

ANSI output is already modelled as styled spans. `OutputView` renders styled runs (foreground/background/bold) today. An `output.line` message with a `spans` array maps onto the existing line model — there is no need to parse ANSI a second time for the wire.

The HUD panel spec is already a serializable UI description. Plugins build panels declaratively:

```
scrye.addPanel{
  title = "Viking Status", accent = "#46B45A",
  widgets = { {type="gauge", ...}, {type="barlist", ...}, {type="buttonrow", ...} }
}
```

Buttons, gauges, `barlist`, `colorgrid`, `input`, `value`, `label`, `progress`, `text` — these are *data*, not pixels. If the companion streams the `PanelSpec` and the phone renders it natively, then custom command buttons, status gauges, refinery bars, travel lists, and auto-trader controls all cross over for free, and **every plugin written from now on automatically gains a mobile UI**.

This is the most important architectural consequence in the whole document: the companion is not a second interface to design and maintain in parallel — it is a **second renderer of specs the desktop already emits**. The protocol should be built around that from day one.

3. Message model

Structured messages, never screenshots. JSON over WebSocket for the live channel; a small request/response surface (over the same socket or plain HTTPS) for setup and history.

Output line (note the batching guidance in §3.1):

```
{
  "type": "output.line",
  "sessionId": "threescapes-eldran",
  "sequence": 18422,
  "timestamp": "2026-08-01T17:58:31+02:00",
  "spans": [
    { "text": "The wiremouth ", "fg": 1 },
    { "text": "attacks you!", "fg": 15, "bold": true }
  ]
}
```

State update:

```
{ "type": "state.update", "sessionId": "threescapes-eldran",
  "path": "character.health", "value": 812, "maximum": 1000 }
```

Command from the phone:

```
{ "type": "command.send", "sessionId": "threescapes-eldran", "command": "north" }
```

HUD panel spec (streamed, then updated in place):

```
{ "type": "hud.panel", "sessionId": "...", "panelId": "viking-status",
  "spec": { "title": "Viking Status", "accent": "#46B45A", "widgets": [ ... ] } }
{ "type": "hud.state", "sessionId": "...", "panelId": "viking-status",
  "bind": "vik.refinery", "value": [ ... ] }
```

Session state:

```
{ "type": "session.state", "sessionId": "...", "connected": true,
  "character": "Eldran", "world": "ThreeScapes", "room": "Megacity" }
```

3.1 Throughput matters

3Scapes combat can emit hundreds of lines per second. Do **not** send one WebSocket frame per line with a full span array each. Two mitigations:

- **Batch/coalesce** output lines within a short window (~50–100 ms) into a single frame.
- **Palette-index spans.** Send an ANSI/theme palette *index* (`"fg": 1`) rather than a hex string per span; the phone resolves it against the palette it was handed at pairing. Fall back to hex only for true-color spans.

3.2 Suggested C# contract (`Scrye.Companion.Protocol`)

```
public sealed record OutputLineMessage(
    string SessionId, long Sequence, DateTimeOffset Timestamp,
    IReadOnlyList<OutputSpanDto> Spans);

public sealed record OutputSpanDto(string Text, int Fg, int Bg, bool Bold);

public sealed record StateUpdateMessage(
    string SessionId, string Path, double Value, double? Maximum);

public sealed record SendCommandMessage(string SessionId, string Command);

public sealed record SessionStateMessage(
    string SessionId, bool IsConnected, string? CharacterName, string? WorldName);

public sealed record HudPanelMessage(string SessionId, string PanelId, PanelSpecDto Spec);
```

These DTOs are the one contract both sides code against, and the only genuinely new shared code the project needs.

4. Integration seams in the current codebase

The companion touches Scrye at a small number of well-defined points:

- **Output out:** tap the same line model `OutputView` consumes; assign each line a monotonic `sequence` as it is appended (the ring buffer for resume — see §6 — reuses the existing Replay / capture / logging buffers).
- **State out:** subscribe to the shared state tree (`character.*`, `vik.*`, `plugin.<id>.*`) — the same channel `scrye.watch` already exposes.
- **HUD out:** serialize `PanelSpec` on panel build and stream `setState` bindings as `hud.state` deltas.
- **Commands in:** phone `command.send` must land on the **same hook as typed input** — `WorldViewModel.SubmitCommand` / `ReceiveBroadcast` — so aliases, triggers, highlights, and logging apply exactly as if the command had been typed at the desktop. The phone never bypasses the command pipeline.
- **Sessions:** the companion enumerates `MainWindowViewModel.WorlDs`; switching the active session on the phone just changes which `sessionId` it subscribes to. The desktop keeps every world connected regardless.

5. Transport tiers

Three ways to connect the phone to the PC, in the order they should be adopted.

Local network (first version). Phone → `192.168.x.x:port` → desktop, over Wi-Fi. Fast, no external server, no account. Works while both devices share a network; may need a firewall allowance. This is the right first target.

User-run mesh VPN (best for remote). A private WireGuard/Tailscale mesh makes the phone and PC behave as if on the same LAN, from anywhere, with no public exposure of the desktop and no service for us to operate. From Scrye's perspective it is still just a private IP. **This is the recommended remote-access path** for anyone comfortable installing Tailscale.

Hosted relay (deliberately deferred, probably never built by us). Both ends dial out to a relay that bridges them. It "just works" from anywhere and enables push — but it is not a feature, it is a *service*: user accounts, uptime, security liability, bandwidth, and (for EU users) GDPR responsibility over other people's private MUD chat. See §7.2 for the one narrow reason a relay might still be justified.

6. Reconnection & resume

Every output event carries a monotonic `sequence`. The phone remembers the last sequence it received. On reconnect:

```
{ "type": "session.resume", "sessionId": "threescares-eldran",  
  "lastReceivedSequence": 18422 }
```

The desktop replays everything after that point from its ring buffer. If the gap is larger than the buffer, it sends a **snapshot** instead — latest N lines, current variables, current room, character state, active timers, command shortcuts, and current HUD panels — and the phone rebuilds from scratch. Because the PC never dropped the MUD connection, "the phone was asleep for an hour" is just a large resume gap, not a lost session.

7. Security

The companion can send commands into a live session, read private tells, and see login state. It is a real attack surface and is designed as one.

Baseline for every build: encryption on the wire, device pairing, authentication on every connection, revocable per-device keys, rate limiting, per-session permissions, **no unauthenticated listener**, and a visible log of connected devices. The mobile app should generally **never receive the saved MUD password** — the desktop handles login and only ever exposes the *active* session.

Per-device permissions, e.g.:

Joakim's iPhone may:

- ✓ View output ✓ Send commands ✓ Run aliases
- ✓ Switch sessions ✓ View maps
- x Edit scripts x Install plugins x Change saved passwords

7.1 Pairing

1. Open "Mobile companion" in the desktop client.
2. Desktop shows a QR code containing: host address, a short-lived pairing token, the desktop's **public key / TLS cert fingerprint**, and the protocol version.
3. Scan with the mobile app.
4. Approve the device on the PC.
5. Devices exchange keys; the phone **pins** the desktop cert from the QR.

Because the QR carries the cert fingerprint, a **self-signed certificate is completely fine** — the phone pins it at pairing time, so there is no CA problem and no plaintext LAN traffic. Use TLS even on the LAN. Bind the listener to the LAN/loopback interface; never `0.0.0.0` by default. Prefer device-specific keys with revocation over a single shared password.

7.2 The one thing that forces a server: background push

A VPN keeps the *socket* alive but the mobile OS still suspends the *app*. So "a tell arrived while your phone was asleep and Scrye wasn't open" cannot be delivered over LAN or VPN alone — it requires APNs/FCM and therefore a small push service. The decision is therefore not "relay or not" but **"is push-while-closed a must-have?"** If no, the whole accounts/hosting/liability burden disappears. If yes, build the *smallest possible* push relay and route **only** notifications through it — never the session traffic.

Notifications the desktop could raise (delivered live while the app is connected; push-while-closed needs §7.2): tell received, character disconnected, low health, script paused, route completed, login finished.

8. Frontend: browser first, native maybe later

8.1 Browser companion is the MVP

Because Scrye is .NET, the desktop can host Kestrel + a WebSocket endpoint **in-process** trivially, and serve a small single-page app. A browser frontend:

- validates the entire protocol *and* the HUD-spec-streaming idea end to end,
- runs on both iPhone and Android with no app-store step,
- gives a genuinely usable phone UI in days, not months.

The Telnet restriction never applies to the browser, because the *PC* makes the Telnet connection; the browser only speaks HTTPS/WebSocket to the PC. **Live in the browser**

version for a few weeks before deciding native is worth it.

8.2 If/when native: Avalonia mobile, not MAUI

The instinct is that MAUI suits a thin companion. For *this* codebase the opposite holds: Scrye already has custom Avalonia HUD controls (`BarListView`, the dim-ramp gauge, `colorgrid`). Avalonia-mobile lets the companion **reuse those renderers** and share the protocol DTOs; MAUI would mean rebuilding every one of them. So Avalonia-mobile is the cheaper native path here — but only after the browser version has proven what native actually needs to add (better keyboard, notifications, offline cache, device integration).

8.3 Mobile UX the structured feed unlocks

Custom command pads (`N S E W U D`, `Attack / Defend / Flee / Heal`), swipe actions (up = previous command, left = map, right = chat), a **chat-only screen** (tells/says/ channels with the room and combat spam filtered out), and voice-to-command ("go north", "tell Erik I'll be there soon") — all of which are just different views over the same structured stream, with the desktop still deciding how aliases and command processing work.

Reference mobile layout:

```
| Eldran      HP 82%  ⋮ |
|-----|
| MUD output |
|-----|
| > command  |
|-----|
| N  S  E  W  Look  Attack |
|-----|
| Output  Chat  Map  Status |
|-----|
```

Tablets can use a wider two-panel layout.

9. Project structure

Resist a big reorg of the existing assemblies. The genuinely new pieces are only two:

<code>Scrye.Companion.Protocol</code>	NEW — shared DTO records (§3.2); referenced by app and phone
<code>Scrye.Companion.Server</code>	NEW — Kestrel + WebSocket host; runs INSIDE <code>Scrye.App</code>
<code>Scrye.App</code>	existing — hosts the server, taps output/state/HUD/commands
<code>Scrye.Core / Scrye.Scripting</code>	existing — unchanged
<code>Scrye.Companion.Mobile</code>	LATER — Avalonia-mobile app (only if native is pursued)

Hosting the server inside `Scrye.App` (rather than a separate process) keeps it next to the `WorldViewModel` / `MainWindowViewModel` state it taps, with no IPC.

10. Recommended build order

1. **Companion protocol** — the `Scrye.Companion.Protocol` DTOs (§3.2).
2. **In-app companion server** — Kestrel + WebSocket inside `Scrye.App`, tapping the seams in §4; LAN-bound, TLS with a self-signed cert.
3. **Browser MVP** — a small SPA rendering output, session state, and streamed HUD panels; sending `command.send`. This is the point the concept is proven.
4. **Pairing & permissions** — QR pairing with cert pinning (§7.1), per-device keys, revocation, the paired-devices list.
5. **Resume & snapshot** — sequence numbers on the output ring buffer + snapshot path (§6).
6. **Remote access** — document the Tailscale/WireGuard path; no service to build.
7. (Optional) **Native app** — Avalonia-mobile, reusing DTOs and HUD controls (§8.2).
8. (Optional, only if push-while-closed is required) **Minimal push relay** (§7.2).

Steps 1–3 are the whole idea, working. Everything after hardens or extends it, and each is independently shippable.

11. Open decisions

- **Is push-while-closed a must-have?** This single answer determines whether a hosted component ever gets built (§7.2).
- **Native at all, or is the browser companion enough long-term?** Defer until after living with the browser MVP.
- **How many concurrent devices / sessions to support in v1?** LAN single-user is the simplest first target.